

**FACULTY
OF MATHEMATICS
AND PHYSICS**
Charles University

MASTER THESIS

František Dostál

**Fitness and novelty in evolutionary
reinforcement learning**

Department of Theoretical Computer Science and Mathematical Logic

Supervisor of the master thesis: Mgr. Roman Neruda, CSc.

Study programme: Mgr.

Study branch: Artificial Intelligence

Prague 2026

I declare that I carried out this master thesis independently, and only with the cited sources, literature and other professional sources. It has not been used to obtain another or the same degree.

I understand that my work relates to the rights and obligations under the Act No. 121/2000 Sb., the Copyright Act, as amended, in particular the fact that the Charles University has the right to conclude a license agreement on the use of this work as a school work pursuant to Section 60 subsection 1 of the Copyright Act.

In date
Author's signature

I would like to thank my supervisor Mgr. Roman Neruda, CSc. for patience and guidance. I would also like to thank my friend Martin Mečiar for moral support when it was most needed.

Title: Fitness and novelty in evolutionary reinforcement learning

Author: František Dostál

Katedra teoretické informatiky a matematické logiky: Department of Theoretical Computer Science and Mathematical Logic

Supervisor: Mgr. Roman Neruda, CSc., Department of Theoretical Computer Science and Mathematical Logic

Abstract: Novelty is a novel approach to modeling selection criteria in evolutionary algorithms and has been proven as viable technique of avoiding pitfalls of false optima in tasks abundant with them, such as solving mazes. Rather than closing the topic however, this finding opened other problems to explore: How to properly apply novelty in tasks that yield slightly better to conventional approaches? How to properly model behavioral space necessary for novelty computation? In this thesis we investigate use of novelty in selected reinforcement learning tasks, its combinations with classical fitness and propose behavior space models for the respective RL tasks.

Keywords: evolution novelty fitness behavioral space

Contents

Introduction	3
1 Introduction to Artificial Agents	4
1.1 Agents and environment	4
1.2 Neural Network Agent	5
2 Environments	6
2.1 Reinforcement learning	6
2.2 Gymnasium	8
2.2.1 Cartpole	8
2.2.2 Lunar Lander	9
3 Evolutionary algorithms	12
3.1 Continuous Optimisation	13
3.1.1 Evolutionary Strategies	13
3.1.2 Differential Evolution	16
3.1.3 Archives	16
3.2 Objective functions	17
3.2.1 Fitness	17
3.2.2 Novelty	18
3.2.3 Combinations of Novelty	19
4 Comparison criteria and conditions	21
4.1 General criteria	21
4.2 Containers	21
4.3 Individual hyper-parameter selection	22
4.3.1 NN Structure	22
4.3.2 Behavioral Space	23
4.4 Algorithm hyperparameter selection	24
4.4.1 Bayesian Search	24
4.4.2 General Gridsearches	26
4.4.3 Incremental grid search	26
4.4.4 $\mu + \lambda$ incremental grid search	27
4.4.5 DEs incremental grid search	28
4.5 Final comparison	28
4.5.1 Friedman test	28
4.5.2 Wilcoxon signed-rank test	29
5 Experiments	30
5.1 Bayesian Search	30
5.1.1 Iterative grid search	30
5.1.2 Generational grid search	30
5.2 Final comparison	30
5.2.1 Cartpole	30
5.2.2 Lunar Lander	30

5.3 Discussion	30
Conclusion	31
Bibliography	32
List of Figures	34
List of Tables	35
List of Abbreviations	36
A Attachments	37
A.1 Experiment Data Format	37
A.1.1 Single Run Statistics	37
A.1.2 Adaptive Grid Search Protocol	37
A.1.3 Generations Grid Protocol	37
A.1.4 Final Experiment Protocol	37
A.2 Used Hardware	37

Introduction

Evolutionary algorithms are a class of nature based optimisation algorithms predicated on the idea of "survival of the fittest" using the optimisation objective as "fitness", guiding the algorithm's search. However for complex objectives direct use of fitness might clog the algorithm in a false, local, optima, unable to reach true global optima. To solve this problem a variety of approaches have been suggested such as archiving, where variety of individuals is kept by an archive. Or novelty search where fitness is replaced by "novelty", a difference in behaviour from rest of the population. Particularly novelty has been shown to work in constrained reinforcement learning tasks such as mazes. The notion of novelty for reinforcement learning is being even generalised outside of the evolutionary algorithms <citation here>. On the other hand there is a number of tasks where combination with fitness might enhance the novelty search, suggesting local competition like MAP-ELites or Dominated Search or other k-neighbours algorithm. These techniques however tend to be expensive for computation.

In this thesis we will characterise two reinforcement learning environments, Carpole and Lunar lander, suggest interpretable behavioral representation for them and apply different evolutionary algorithm techniques, such as differential evolution, itself being a mechanism to introduce controlled diversity into the population. We will compare several lightweight applications of novelty combined with fitness, showing their viability and pitfalls on the tasks at hand and suggest best approach for each of the environments. In the chapter 1 we will introduce terminology pertaining to agents we will be optimising. In chapter 2 we will introduce the aforementioned reinforcement learning tasks and detail the optimising algorithms in chapter 3. In chapter 4 we will review background of the experiments and finally in chapter 5 we will discuss the results of our experiments.

1. Introduction to Artificial Agents

To understand the reasoning guiding this work we have to first look at the fundamental subject of our efforts - the artificial agents.

We will look at their structure, guiding principles, practical implementation and nor last nor least, their use.

1.1 Agents and environment

All agents exist in environments that provide context to their agency. An agent gets information about the **environment** it finds itself in (e.i. **percepts**) through **sensors** and acts on the environment through **actuators**. Both actuators and sensors are usually characteristics of the agent. Another characteristic of an agent would be the setup of its internal machinations.

Definition 1 (Russell and Norvig [2021]). *Let E be an environment where agent G is located. G is equipped with actuators A and sensors P and so we can identify $obss_P(E)$, a space of all possible sequences of percepts available to G within E and $A(E)$ space of available actions in E . Then function $G : obss_P(E) \rightarrow A(E)$ is called an agent function, defining behaviour of the agent G in environment E .*

We illustrate the relationship of these concepts in the diagram 1.1. Note that the provided definition of agent policy is characteristic of deterministic agents. For agents in stochastic environments we need to reframe the agent function as probabilistic policy. We will discuss this reframing further in section 2.1.

Because we want our agents to solve some tasks in their respective environments with certain efficiency i.e. we want them to be intelligent, we need to define a notion of rationality that can help us better capture what we mean by intelligence. First however we need to measure how useful or detrimental a situation can be for our agent.

Definition 2 ([Russell and Norvig, 2021]). *Be that E is an environment and $seq(E)$ is a space of all possible sequences of states within E . Then function $f : seq(E) \rightarrow \mathbf{R}$ is a performance measure of some agent operating in E .*

Definition 3 (Russell and Norvig [2021]). *For each possible percept sequence, a rational agent should select an action that is expected to maximize its performance measure, given the evidence provided by the percept sequence and whatever built-in knowledge the agent has.*

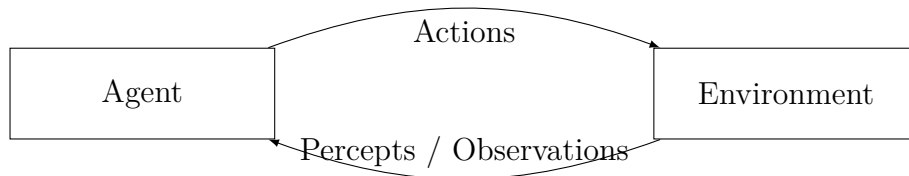


Figure 1.1: Agent and environment relationship

We should note that in the real-world applications the rationality of an agents can often be limited by their capacity to store the evidence gathered from percepts, by the lack of their built-in knowledge or the computational capacity to estimate action sequence that can be expected to achieve global maximum of the agent's performance measure.

This is often characterised as a capacity to form *belief*. With that we expand on the notion of agent policy In this thesis we are more concerned with the properties of the evolutionary algorithms rather than evolution of complex agents. Therefore we consider only *reflexive agents*.

Definition 4 ([Russell and Norvig, 2021]). *An agent is called simple reflex agent if it takes into account only the current percepts, never keeping any history or state information during it's run.*

1.2 Neural Network Agent

The concept of neural networks (NN) comes from an area of machine learning called deep learning that was conceived to attempt to model biological neuron activity. Since it has become one of the most successful branches of machine learning (Russell and Norvig [2021]). The neural networks consist of neurons organised in layers.

Definition 5. *Let $W \in R^{m \times n}$ and $b \in R^n$ and let $f : R^n \rightarrow R$. Then we call W the weights of a neuron layer with n neurons, b the bias of the layer and f the activation function of the layer when given an input vector $x \in R^m$ we calculate the output of the layer as $f(W^T x + b)$. Two layers have directional connection when the output of first layer is used as input of the subsequent layer.*

The purpose of the activation function is to introduce exploitable non-linearity into the neural system. We recognise several basic activation function.

- sigmoid

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

- tanh

$$\tanh(x) = \frac{e^{2x} - 1}{e^{2x} + 1}$$

- ReLU

$$ReLU(x) = \max(0, x)$$

Definition 6 (Russell and Norvig [2021]). *A neural network where the connections between layers of neural network form an acyclic graph is called a feed-forward neural network.*

Because the feed-forward NN doesn't hold any state or holds onto any previously acquired results and only produces output for a given input, they are ideal for implementation of simple reflex agents. In this thesis we use feed-forward networks exclusively, so any further mention of neural network should be assumed to be feed-forward.

2. Environments

In this chapter we will discuss the environments we have chosen to use in our examination of evolutionary algorithms and the theoretical background that will help us analyse them.

2.1 Reinforcement learning

Reinforcement learning might be best introduced by description of the problem it aims to solve.

Definition 7 (Markov's decision problem). *Let P be a probability measure over the states of the environment. We define Markov's decision problem (MDP) by a four-tuple (S, A, T, r) where:*

- S is a set of all possible states of the system, or state space
- A is a set of all possible actions available in the system or action space
- T is a transition function:

$$\forall s', s \in S, a \in A : T(s, a, s') = P(s'|s, a)$$

- $r : S \times A \times S \rightarrow \mathbf{R}$ is reward function, $r(s, a, s')$; being the reward received when the execution of action a in state s leads to state s' .

To tie this into the information from chapter 1 $A = A(E)$, R is performance measure of an agent solving the MDP and the percepts have bijection to real states of the environment. This would not be the case for partially observable MDP.

Definition 8 (Kaelbling et al. [1998]). *Let P be a probability measure over the states of the environment. We define Partially Observable Markov's decision problem (POMDP) by a four-tuple (S, A, Ω, T, O, r) where:*

- S is a set of all possible (true) states of the system, or state space
- A is a set of all possible actions available in the system or action space
- Ω is a set of all possible observations on the system, or observation space
- $T : S \times A \times S \rightarrow [0, 1]$: is a transition function over true states.

$$\forall s', s \in S, \forall a \in A : T(s, a, s') = P(s'|s, a)$$

.

- $O : S \times A \times \Omega \rightarrow [0, 1]$.

$$\forall s' \in S, \forall a \in A, \forall o \in \Omega : O(s', a, o) = P(o|a, s')$$

or the probability of making observation o given that the agent took action a and landed in state s' .

- $r : S \times A \times S \rightarrow \mathbf{R}$ is reward function same as in MDP.

Now that we have characterised the environments we can apply to them concepts we have revied in the chapter 1. We begin by specifying the notion of percept sequence space from definition 1.

Definition 9 (Ross et al. [2014]). *Let Ω the space of all observations ($\Omega \cong S$ for MDP), A be the action space, $o_t \in \Omega$ be observation at time t and $a_t \in A$ action taken by the agent in time t . Then*

$$h_t = (a_0, o_1, \dots, a_{t-1}, o_t)$$

is a history at time t .

Definition 10 (Ross et al. [2014]). *Let P be probability measure over states of the environment S and $s_t \in S$ be the state of the environment at time t . Then function $b_t : S \rightarrow [0, 1]$ such that*

$$b_t(s) = P(s_t = s | h_t, b_0)$$

is called a belief state of an agent, where b_0 is the apriori belief state and h_t is history at time t .

We also need to reframe the agent function discussed in definition 1.

Definition 11 (Ross et al. [2014]). *Let P be a probability measure and A be an action space. Then function π defined as $\pi(b_t, a) = P(a | b_t)$ where $P(a | b_t)$ denotes probability of doing action a given belief state b_t is called agent policy.*

Definition 12 (Ross et al. [2014]). *Let r be a reward function, A an action space, T a transition function, $\gamma \in [0, 1)$ a discount factor and Π be a set of all possible agent policies, then*

$$\pi^* = \arg \max_{\pi \in \Pi} \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t \sum_{s \in S} b_t(s) \sum_{a \in A} \sum_{s' \in S} r(s, a, s') T(s, a, s') \pi(b_t, a) \mid b_0 \right]$$

is called optimal policy.

The goal of reinforcement learning is then to proximate optimal policy by trial and error, based on the received reward, as accurately as possible. This can be done utilising various online or ffline methods such as Q-learning. In the literature and specifically in the cited Ross et al. [2014] the the definition 12 features γ as an discounting factor for the reward. This is a crucial parameter for reinforcement learning algorithms that require continuous rewards. However as we will discuss further in this chapter and chapter 3, in evolutionary algorithms we evaluate the policy as a whole and are only presented with final value calculated. Therefore it is not nessessary to know or discover particular value of γ for the environments presented in this thesis.

Note that since in this thesis we are only concerned with simple reflexive agents, the belief state is not explicitly calculated statistic but is nonetheless reflected in the learned reactions to observation.

2.2 Gymnasium

We intend to set our agents into environments provided by Gymnasium project which emerged from it's predecessor OpenAI Gym project. Both of them Python libraries, trying to provide researches with benchmark reinforcement learning problems in form of simple games, sometimes even recreating old console games such as Air Raid or Backgammon.

While Gymnasium provides wide range of environments, quite a number of them if not most do not provide their outputs in structured form. Instead the resulting screen has to be processed by user defined visual processor to extract information crucial for completing the task.

This introduces a layer of complexity that is outside of the scope of this thesis. Therefore we have decided to narrow our selection only to environments which do provide structured output. We describe the loop of receiving and providing feedback between agent and environment in algorithm 1.

Algorithm 1: Environment Interaction Loop

```
Input: Environment  $\mathcal{E}$ 
Input: Number of episodes  $n$ 
Input: Observation aggregation functions  $agg$  and  $transform$ 
 $R = 0$ ;
for  $i = 0$  to  $n$  do
     $o \leftarrow \text{reset}(\mathcal{E})$ ;
     $O \leftarrow \text{reset}(O)$ ;
    while not terminated do
         $a \leftarrow \text{Agent}(o)$ ;
         $(o, r, \text{terminated}) \leftarrow \text{Environment}(a)$ ;
         $R \leftarrow R + r$ ;
         $O \leftarrow agg(O, o)$ ;
    end
end
return  $R/n, transform(O)$ ;
```

As mentioned in the introduction the selected environments are called Cartpole and Lunar Lander.

We will discuss the full purpose of agg and $transform$ in the section 3.2.2 and section 4.3.2.

2.2.1 Cartpole

Cartpole is one of the most used benchmark environments in the Gymnasium library. It's a simple environment where the agent controls a cart rolling on flat surface with a pole on top. The goal is to keep the pole balanced on top of the cart. The reward is calculated every step depending on the angle of the stick. This means we have bounded total reward between 0 and 500 points.

Definition 13. *An agent that achieves score 500 in Cartpole is called solution or solving individual.*

Observation space

Cartpole observation space is expressed as a 4-dimensional vector. We deliniate possible values of the vector in 2.1. Note that for Cartpole enviroment this observation space reflects full state of the enviroment in the given moment and therefore it could be characterised as MDP rather than POMDP.

Index	State Variable	Min	Max
0	Cart position	-4.8	4.8
1	Cart velocity	$-\infty$	∞
2	Pole angle	-0.418 rad	0.418 rad
3	Pole angular velocity	$-\infty$	∞

Table 2.1: Observation space of Cartpole

Action space

The action space is discrete with only two values possible as presented in the table 2.2. Note that we cannot chose to do nothing.

Action	Description
0	Push cart to the left
1	Push cart to the right

Table 2.2: Cartpole Action Space.

2.2.2 Lunar Lander

The other environment we have selected is Lunar Lander where the agent controls descend of a landing shuttle on rugged planetary (or the Moon's) surface with the help of oddly placed rockets. The points are subtracted for staying in the air or for getting the shuttle into a spin. Bonus points awarded for landing in the center of the The shuttle has infinite fuel so the game ends only when the shuttle lands safely or crushes into the ground. Luckily we can at least limit the number of steps for which the environment is allowed to go on. For a neural network training this however poses a specific kind of problem. Firstly very narrow behavioral window that can achieve positive total reward. But mainly as opposed to the Cartpole environment (2.2.1), now the unsuccessful runs may take significantly longer time than the successful ones. This means experiments are particularly costly for slowly converging algorithms. The upper limit of sum total reward is 200, the lower would possibly be $-\infty$, depending on the lenght limitation imposed. We therefore differntiate between several stages of obtaining successful agent.

Definition 14. *An agent that achieves score 200 in Lunar lander is called solution or solving individual.*

Definition 15. *An agent that achieves positive score in Lunar lander is called well-behaved.*

Definition 16. *An agent that achieves average score over 100 in Lunar lander is called very well-behaved.*

Definition 15 stems from the fact that it can only be achieved if the agent does not crash the shuttle as doing so yields a penalisation of -100 points while not obtaining a boost of +100 points for safe landing, driving even a possible solution with score 200 to 0. Similarly for definition 16, it is highly unlikely to average score over 100 without at least once landing with perfect score 200.

Observation space

The observation space of Lunar Lander is represented as 8 dimensional vector. Deliniation of its values can be found in the table 2.3. Note that while the observation space is quite descriptive it does not capture all the information about enviroment’s state directly such as wind velocity or proximity to the ground. Because the landscape (except for a landing pad in the center) might be different in each episode, we can only infer their parameters from the observations (such as velocity and leg contact with ground) indirectly. For simplicity we do not allow wind in our episodes as is the default setting of the environment. This means that the distiction between observation and true state becomes most tangible in low altitudes where agent risks either collision with unexpected hill or infinite hovering above unpercieved valley, forgoing the +100 point reward.

Index	State Variable	Min	Max
0	x position	-2.5	2.5
1	y position	-2.5	2.5
2	v_x velocity in x-axis	-10	10
3	v_y velocity in y-axis	-10	10
4	angle θ	-6.2831855	6.2831855
5	angular velocity	-10	10
6	left leg contact (boolean)	0	1
7	right leg contact (boolean)	0	1

Table 2.3: Observation space of Lunar Lander

Action space

Action space for Lunar Lander can come in two shapes. First is discrete where we the agent chooses a number representing one of the engines to ignite. Second is continuous where agent has continuous control over the ignition of the main engine and one of the side engines. This variant allows for more precision during the flight but at the same time makes the policy the agent needs to learn significantly more complex, requiring more resources during the learning phase. To preserve compatibility with Cartpole setup and considering feasibility of multiple experiments ahead we will work with the discrete version of the action space outlined in table 2.4.

Action	Description
0	Do nothing
1	Fire left orientation engine
2	Fire main engine
3	Fire right orientation engine

Table 2.4: Action space definition for the control system.

3. Evolutionary algorithms

The evolutionary algorithms (EA) are a class of optimisation methods modelled after our ideas of biological process of evolution. This chapter is based primarily on Introduction to Evolutionary Computing from Eiben and Smith [2003].

Definition 17. *General maximising optimisation problem takes the form:*

$$\begin{aligned} \text{maximise: } & f(x) \\ \text{subject to: } & g_i(x) \leq 0, \quad i = 1, \dots, m, \\ & h_j(x) = 0, \quad j = 1, \dots, p, \end{aligned}$$

where x is the solution vector, f is objective function, and g_i, h_j are constraints imposed on x

Although theoretically it could be also minimising, in this thesis we will not encounter the minimising optimisation and so our concern is mainly with the maximising case.

Because of this biological connection solution vectors are in the literature often referred to as individuals and we will refer to them this way henceforth as well. Alternatively when emphasising either the numerical form of the vector or semantic interpretation of this form beyond the optimisation algorithm the individual may be referred to as either a **genotype** or **phenotype** respectively.

Group of individuals representing candidate solutions at a given time is referred to as a population. In general case evolutionary algorithms then evolve the population by applying three operations (operators) to it in a loop until either the population converges to optimum or the limit on number of steps is reached.

Definition 18. *Let f be objective function in minimisation (resp. maximisation) problem. We call an individual i more successful than individual j if value $f(i) < f(j)$, resp. $f(i) > f(j)$.*

Algorithm 2: Generalised Evolutionary Algorithm

Data: Initial population = pop
Data: N = Number of generations
Result: Optimised population = pop
for $g \in 1..N$ **do**
 $co \leftarrow \text{Crossover}(pop);$
 $m \leftarrow \text{Mutate}(co);$
 $pop \leftarrow \text{Select}(m);$
end
return pop

Let us review the meaning of the aforementioned operators.

- crossover usually involves recombination of features from two or more individuals from the population. Its purpose is to provide means of exploitation of features beneficial to the population already developed by some individual.

- mutation provides means of exploration by introducing randomness into the individuals EA without crossover would be more akin to a random walk algorithm.
- selection then is about filtering the resulting individuals based on the objective function of the algorithm. It might be performed deterministically or with varying degree of stochasticity as elaborated in section 3.1.

It should be noted that the order of mutation and cross-over here are arbitrary and are specific to each particular evolutionary technique. For further discussion of selection process we need to introduce a few terms specific to evolutionary algorithm that have been borrowed from their counterpart in biology.

3.1 Continuous Optimisation

Historically the individuals optimised by EA used be primarily conceived as vectors over binary domain - $\{0,1\}$. However this approach was later expanded into the domain of vectors over real numbers. In literature this is referred to as continuous optimisation. In this thesis we are interested in neuro-evolution, a subset of continuous optimisation as neural networks discussed in the chapter 1 are represented as a series of vectors of real numbers. But since the intent of this thesis is mainly to compare different objective functions we will not delve too deeply into the intricacies of neuro-evolution field and instead approach the following techniques from the more general standpoint of continuous optimisation.

In this section we will review the selected evolutionary algorithms and describe them as they are going to be used during the experiments in chapter 5.

3.1.1 Evolutionary Strategies

We base information in the section mainly on the [Beyer and Schwefel, 2002] paper. Evolutionary strategies (ES) are very direct incarnation of algorithm 2 into the realm of real numbers.

We recognise two types of evolutionary strategies: $\mu + \lambda$ and μ, λ . μ denotes size of already existing population while λ denotes size of new population pool. In $\mu + \lambda$ selection happens from the existing population pool and the new population pool at the same time while the μ, λ variant only ever selects from the new population pool. This makes the evolution $\lambda + \mu$ more stable as established successful individuals have a bigger chance to pass to the next generation unless a more successful individual has been found. That said, for some problems this approach risks stagnation and it is preferable to use μ, λ . However the experience with our selected environments from chapter 2 clearly showed dominance of $\mu + \lambda$ as even for the Cartpole, simpler of the environments, improving mutations were too costly to be lost on an unlucky new generation.

Therefore all further discussion of evolutionary strategies will be focused on $\mu + \lambda$. We detail the procedural aspects of $\mu + \lambda$ in the algorithm 3.

Mutation

Distinct feature of evolutionary strategies is the mutation operator that takes as a parameter standard deviation σ and which mutates the value stochastically by a number $y \sim N(0, \sigma)$. In the literature we can often find ES where the mutation σ is part of the individual genotype this might help the evolution as more successful individuals might enforce milder mutation while less successful individuals might mutate wildly in search of better genotype. For our purposes this however introduces two problems. Firstly it complicates interpretability of the results and since we are more interested in the comparison than raw results we cannot ignore this downside. Secondly this approach intuitively works well for cases where objective function is also the meta-measure we actually care about but we are precisely interested in comparison with cases where the objective function is rather orthogonal to actual usefulness of the individual as we will discuss in section 3.2. There this approach could be chaotic at best and detrimental at worst.

For these reasons we opt for a global σ that does not change during the run of the evolution.

Crossover Methods

Crossover is a method of combining values from two selected individuals. Based on We will discuss mainly two kinds of cross-over:

- **Arithmetic** crossover where we combine genotype vectors x_1, x_2 into a new genotype x' as follows:

$$x' = \frac{x_1 + x_2}{2}$$

- **Uniform** crossover where iterate over components of two parent genotype vectors x, y through index $i \in 1 \dots d$ where d is the size of the genotype (we assume both parents are the same length). Then with parameter p_u we produce new individual's genotype v like so:

$$v_i = \begin{cases} x_i & \text{if } r \sim \text{Uniform}(0, 1) < p_u \\ y_i & \text{otherwise} \end{cases}$$

We do not consider position based cross-over methods as they might become quite difficult to manage for the neuroevolution purposes.

Tournament selection

Selection in evolutionary algorithms can be realised by various means such as roulette selection where each individual i has likelihood of passing into the next generation p_i as follows: $p_i = \frac{f(i)}{f(pop)}$, where $f(i)$ is objective function value for the individual i and $f(pop)$ is sum of objective value of all individuals. This however works only for objective functions that give out only either negative or positive numbers. As we have described in chapter 2 environments explored in this thesis can give out rewards with value negative or positive based on the agent's performance. Therefore we will utilise tournament selection described in

Goldberg and Deb [1991]. Tournament selection works on simple principle where we select pool of individuals, size of which is called *tournament size*. From them we then select the one with the most optimal objective function value [Goldberg and Deb, 1991]. We detail the workings of tournament selection in algorithm 3. Note that for the purposes of this thesis we have selected *tournament size* = 3, or ternary tournament. Bigger tournament size provides greater stability to the search as the more optimal individual are given more chances to be selected. The ternary tournament has been selected on empirical basis specifically for our environments.

Algorithm 3: Evolutionary Strategy $\lambda + \mu$

Input: Population size λ , new generation pool size μ , crossover rate CR , mutation rate MR , objective function $f(x)$, crossover operator $CrossOp(x)$, mutation power σ , number of generations G

Initialize population $\{x_i^0\}_{i=1}^\mu$;

Evaluate $f(x_i^0)$ for all i ;

// x^0 forms initial population

for $g = 0$ **to** $G - 1$ **do**

for $i = 1$ **to** λ **do**

 // Crossover:

 randomly choose $r_{CR} \sim Uniform(0, 1)$;

if $r_{CR} < CR$ **then**

 Select random individuals $a, b \sim Uniform(1, \mu)$;

$$v_i \leftarrow CrossOp(x_a^g, x_b^g)$$

else

 Select random individual $a \sim Uniform(1, \mu)$;

$$v_i \leftarrow x_a^g$$

 // Mutation:

 randomly choose $r_{MR} \sim Uniform(0, 1)$;

if $r_{MR} < MR$ **then**

$$v'_i = v_i + \sigma \mathcal{N}(0, 1)$$

else

$$v'_i = v_i$$

 Evaluate $f(v'_i)$;

 // v' forms the new population pool

$v'_i \rightarrow x_{\mu+i}^g$;

 // Tournament selection:

for $i = 1$ **to** μ **do**

 Select random individual $a, b, c \sim Uniform(1, \mu + \lambda)$;

$$x_i^{g+1} \leftarrow \arg \max(f(x_a), f(x_b), f(x_c))$$

return x^G

3.1.2 Differential Evolution

In the introductory paper Storn and Price [1997] on which this section is mostly based, the differential evolution is characterised mainly by its industrial capabilities such as easy parallelisation and application.

Because of these benefits it is a popular technique in the field of neuroevolution. We detail the whole process in algorithm 4. The algorithm for resolving a single generation happens in a single loop with the familiar stages which we detail in algorithm 4.

Selection

Unlike in evolutionary strategies described in section 3.1.1, selection is done on individual basis to enhance parallelisation capabilities of the algorithm. This means that for each individual x_i we either select it or its proposed replacement v_i based on the values of objective function for each of them. Of course we keep the more successful one.

Mutation

While Evolutionary strategies introduce stochasticity in explicit and controlled manner, differential evolution relies on a special form of contextual mutation where three individuals from the population are drafted and combined. This means that we do not have to search for optimal value of σ .

We select three individuals x_a, x_b, x_c distinct from x_i and compute a new individual: $x'_i = x_a + \alpha(x_b - x_c)$ where x_a is called anchor, $(x_b - x_c)$ differential and α is scaling factor.

Although the scaling factor α does not have probabilistic interpretation, in some sense it corresponds to mutation rate in ES as the greater it is, the more stochasticity affects the system.

Recombination

During recombination we decide for each vector component of the newly forming individual x'_i whether we will source it from the already mutated version x'_i or the original x_i based on recombination rate γ . Recombination is rough translation of the concept of crossover for the purposes of single easily parallelised loop and recombination rate could be likened to the uniform crossover cross-rate R_U .

3.1.3 Archives

A technique that may enhance the techniques presented in this sections is installing an archive into the algorithm. Archive is a longterm storage outside of the optimised population of a certain size where a batch of individuals selected from regular population is added periodically and are being reintroduced during the selection. This approach can help preserve diversity of individuals or on the contrary to preserve the best solutions found so far. We will distinguish archives intended for the second purpose as *elite archives*.

Algorithm 4: Differential Evolution

Input: Population size N , scaling factor α , recombination probability γ ,
objective function $f(x)$, number of generations G
Initialize population $\{x_i^0\}_{i=1}^N$;
Evaluate $f(x_i^0)$ for all i ;
for $g = 0$ **to** $G - 1$ **do**
 for $i = 1$ **to** N **do**
 // Mutation:
 Select random individuals $a, b, c \neq i$;

$$v_i = x_a^g + \alpha(x_b^g - x_c^g)$$

 // Crossover:

$$u_{i,j} = \begin{cases} v_{i,j} & \text{if } r \sim \text{Uniform}(0, 1) < \gamma \\ x_{i,j}^g & \text{otherwise} \end{cases}$$

 // Selection:
 Evaluate $f(u_i)$;
 if $f(u_i) \geq f(x_i)$ **then**
 $x_i^{g+1} \leftarrow u_i$;
 else
 $x_i^{g+1} \leftarrow x_i^g$;
 return x_i^G

For evolutionary strategy we simply add the archive batch from which a new generation pool is being selected as this will not affect the run of the algorithm in any way.

For differential evolution we cannot just add the archive to the existing population. We actually have to reintroduce the archived individuals instead of parts of the population. In this thesis we will do this on basis of random pick from the *population + archive*.

3.2 Objective functions

In this section we will discuss possible objective functions for the evolutionary optimisation search.

3.2.1 Fitness

Using objective function as the utility function of the algorithm is the most direct way of establishing relationship between the solution we want and the individuals in the evolving population.

Though this approach ensures that the most optimal individuals from given population will generally be selected into the next generation during the evolution it does not guarantee that their potential for evolving is not diminished either. This way the evolution using fitness can easily get stuck in local optimum

of the objective function. While in most cases it might be still possible to reach global optimum on the given problem by changing the hyper-parameters to favour random exploration, it may come to point where the benefits of evolutionary approach are made void.

Archiving

As stated in section 3.1.3 for fitness the two main reasons for introducing archive into an fitness based EA is either to promote diversity by randomly preserving part of the population which might have been eradicated to the detriment of the overall evolution or on the contrary to preserve the best performing individuals so they may not be accidetally selected out of the gene pool.

3.2.2 Novelty

In its introductory paper novelty Lehman and Stanley [2011] is conceived as a guide of the evolutionary search to help avoid traps of local optima.

Definition 19 (adapted from [Doncieux et al., 2019]). *Let E be environment and h_t be a history up to a time t and let $d(x, y)$ be a metric over set B . Then (B, d) is a behavioural space if there is a function $\omega_B : h_t \rightarrow B$, called observer function.*

For the purposes of this thesis the observer function is implemented through the $agg(x)$ and $trasnform(x)$ functions used in algorithm 1.

Definition 20 (adapted from [Lehman and Stanley, 2011]). *Let P be a population of individuals within EA and (B, d) be a behavioural space. Then for each individual a we define novelty:*

$$N(a) = \frac{\sum_{b \in P, b \neq a} d(a, b)}{|P| - 1}$$

With novelty as objective function the selection process incentivises diversification of behaviour among the population. This allows the evolutionary search to retain variety within the population across generations, leveraging the crossover as well as mutations to leave local optima.

Downside of this approach might be in the ambiguity in the choice of behavioural space. Incorporating too many environment variables will lead to faux-exploration in variables completely orthogonal to fitness. On the other hand leaving out an important variable means the algorithm will not retain diversity where it matters, making novelty pointless. This means the behavioural function and space have to be selected empirically for given problem.

If we do not state otherwise in this thesis novelty will be calculated in relation to the upcoming generation $g + 1$ During the unrecorded preliminary experiments we have not found any significant advantage to other methods of computation.

Archiving

Unlike in fitness described in section 3.2.1, we will not use elite archive for the novelty search, instead we will discuss our elitist solution as a combination of fitness and novelty in section 3.2.3.

In fitness the archive had a role of providing variety, which is the main goal of novelty search. Here we actually hope to use the randomness as a stabilising element since reintroduction of old behaviours will propt the new generation to avoid them. This approach was with success used in Gomes et al. [2015], outperforming pure novelty and novelty based archive.

3.2.3 Combinations of Novelty

The novelty does not take into account the individual's performance with objective function. Therefore when two individuals with similar behavioural characteristics appear but one of them performs significantly better than the other, pure novelty algorithm can easily select the less performant individual, losing the more performant genome for further generations. To prevent this we want to meaningfully combine novelty with fitness.

Exponential Decay Weight

Taking inspiration from Cuccu and Gomez [2011], we will try to combine novelty with fitness in the most straight-forward way - linear combination with decaying weights. However first we need to address some issues such as the disproportion in scale of fitness to scale of novelty. This mean we require normalisation for the two. Again taking inspiration from Cuccu and Gomez [2011], we will normalise with the maximum and minimum of both fitness and novelty in the currently existing population for each individual i :

$$\hat{f}_i = \frac{f_i - f_{\min}}{f_{\max} - f_{\min}}, \quad \hat{n}_i = \frac{n_i - n_{\min}}{n_{\max} - n_{\min}}$$

Both maximum and minimum are easy to obtain as they are one $O(n)$ while other methods such as z-score might be computationally more demanding in the multiplicative constant. Then we get combination:

$$c_i = W\hat{f}_i + W'\hat{n}_i$$

Unlike Cuccu and Gomez [2011] we will however decay the weight W of fitness or $W' = (1 - W)$ of novelty over time. So that for base W_b , decay rate r_D , current generation g , total number of generations G and **increasing novelty** we have:

$$W = W_b e^{-r_D \frac{g}{G}}$$

Conversly for **increasing fitness**

$$W' = W_b e^{-r_D \frac{g}{G}}$$

This will allow us to explore whether the exploratory stage has more meaning at the beggining when it generates diverse population that can later pruned by the shift for fitness or is it better to first evolve the population to certain level and then explore with more adept individuals.

Limited Novelty Archive

Another way to combine fitness and novelty is through a special case of archiving such as we proposed in section 3.2.2 but we introduce a fitness threshold T_f so that given a randomly selected batch Q_{rand} and fitness function f we screen the selected individuals so that:

$$Q_{elite} = \{i \in Q_{rand} | f(i) \geq T_f\}$$

Only these screened individuals are then added to the archive. This will help us explore the idea of the archive's stabilisation role.

4. Comparison criteria and conditions

To compare different types of objective functions fairly, we have to select criteria by which we compare them and establish conditions under which will this comparison take place so that our conclusions are fair.

4.1 General criteria

Since fitness provides direct measure of performance we would like to make our comparison on the fitness of the final population and its distribution in it. To that end we have to perform experiments with variety of initialisations of the respective environments, getting more comprehensive picture then from a single run. This will be mainly reflected in the seed for random numbers generator of the environment. Since the promise of the novelty approach lies mainly in avoiding the trap of local optima we also want to review statistics of each generation produced by the algorithm runs

However we also have to take into account the hyper-parameters with which these populations have been achieved. This is especially (but not limited to) population size and number of generations, hyper-parameters responsible for most of the computational complexity of a evolutionary algorithm run.

4.2 Containers

While we have reviewed evolutionary algorithms and proposed different objective functions in chapter 3, we now have to somehow refer to the combination of the two. For this purpose we propose the term **container**. We can divide the containers on the axis of algorithm to ES containers and DE container. And we can divide them on basis of objective function handling:

- **fitness** container implements pure fitness approach as described in section 3.2.1
- **novelty** container implements pure novelty approach as described in section 3.2.2
- **fitness archive** container implements fitness search with random selection archive as described in section 3.2.1
- **elite archive** container implements fitness search with elite selection archive as described in section 3.2.1
- **novelty archive** container implements novelty search with random selection archive as described in section 3.2.2
- **increase novelty** container implements increasing novelty objective as described in as described in section 3.2.3

- **increase fitness** container implements increasing fitness objective as described in as described in section 3.2.3
- **limit archive** container implements novelty search with fitness threshold archive as described in section 3.2.3

Specifically for ES the crossover method is considered a hyperparameter that will be pinpointed through the bayes search described in section 4.4.1.

4.3 Individual hyper-parameter selection

The hyper-parameters characterising individuals are number of layers and the number of neurons in their layers. The choice of these parameters is specific for each task. Generally the more complex is the task the more of either layers or neurons per layer the handling NN requires to be suitable for it.

4.3.1 NN Structure

As discussed in chapter 1 as a choice we are making our agents to be simple feed-forward neural networks. This choice is on account of simplicity but also on account of adequacy as the bigger the neural network is the more resources need to be spent on optimising it. The NN structures are depicted on figures 4.1 and 4.2 for Cartpole and Lunarlander respectively.

Both structures have been selected on account of preliminary experience with the environments. Shapes of input and output layer are determined by the respective observation and action space.

The genome of evolving individual consists purely of the weight and bias matrices of the hidden layers nothing else is subject to evolution. For the purposes of this thesis and in relation to the descriptions of evolutionary mechanics given so far we should think about these matrices as flattened to a single long vector. In the case of Lunar lander with multiple layer the vectors are concatenated.

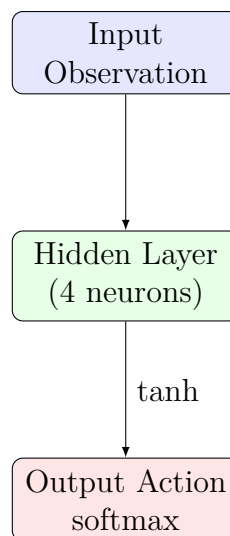


Figure 4.1: Structure of Carpole agentic network

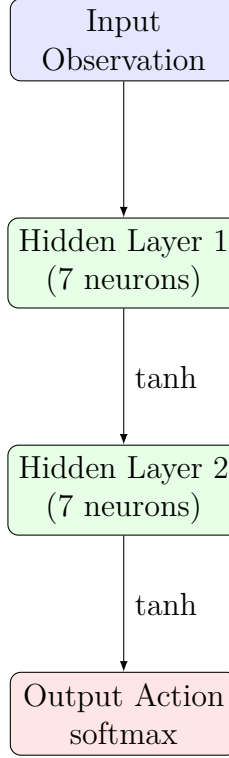


Figure 4.2: Structure of Lunar Lander agentic network

4.3.2 Behavioral Space

As we can see through the definition of observer function in definition 19, the choice of transformation from history to behavioral space can be arbitrarily complex. Recently there have been proposals of automatic behavioral space extraction like in the paper Meyerson et al. [2016] or Salehi et al. [2021]. However the original paper Lehman and Stanley [2011] emphasises application of domain knowledge to the behavioral space design. Because this thesis is in the core comparative and we are interested in the interpretability of the design as much as raw performance, we have opted for manual design based on our preliminary experiments and experience with the environments. We also take into account the fact that the greatest advantage from an automatic behavioral space extraction is obtained mainly in complex environments [Meyerson et al., 2016].

While we won't specify *agg* and *transform* functions introduced in algorithm 1 explicitly for readability purposes, their definitions are implied by respective definition of observer function ω_B and the fact we will only consider the behaviour of the last evaluated episode. While such approach loses some of the information about the evaluated individual it also does not introduce consistency trade-offs of standard aggregations like max, min or *mean* applied component-wise. Their solution is out of the scope of this thesis.

Cartpole

Let $h_{Cart} = (a_0, o_1, a_1, \dots, o_T)$ be history of one completed episode of Cartpole that lasted T steps and o_T is the last observation. Then we define observer

function:

$$\omega_B(h_{Cart}) = (pos_X(o_T), \theta(o_T))$$

where $pos_X(o_T)$ is reported position on x-axis in observation o_T and $\theta(o_T)$ is pole angle both normalised to interval $[-1, 1]$ through their respective maximal values reported in table 2.1.

Lunar Lander

Let $h_{Lun} = (a_0, o_1, a_1, \dots, o_T)$ be history of one completed episode of Lunar Lander that lasted T steps. Then we define observer function:

$$\omega_B(h_{Lun}) = (pos_X(o_T), v_Y(\bar{o}))$$

where $pos_X(o_T)$ is reported position on x-axis in observation o_T , v_Y is a reported velocity in y-axis and $\bar{o}$ is mean of all observations recorded in h_{Lun} . Both components of behavioral vector again normalised to interval $[-1, 1]$ through their respective maximal values reported in table 2.3.

4.4 Algorithm hyperparameter selection

We have reviewed hyperparameters of each algorithm in the chapter 3. Now we will review the process of their selection. The difficulty of the task lies mainly in the complexity of evaluation. As we will discuss later some of the experiments maybe evaluated over day or even several days on the available hardware even for a 100 limited algorithm runs. For this reason we need to utilise bayesian search which is able to prune a lot of the redundant space out so that we may not waste time with it. This however has a considerable downside for our purposes as we would like to compare the algorithms on their best footing. Therefore we refine the bayesian search results with local grid search attempting to deviate from the supposed hyperparameter equilibria.

4.4.1 Bayesian Search

Because we are faced with multiple Bayesian search is an algorithm where we stochastically deliniate where a possibly good solution may lie and then try to exploit this deliniation. Because the specific workings of Bayesian search are time-consuming to implement ad hoc (and also such implementation is out of the scope of this thesis) We will utilise library Optuna for the purpose.

Optuna

Just as in our case a lot of machine-learning techniques need for their proper function to obtain adequate hyperparameters first. Optuna is a Python library designed for this purpose. The library itself is multifasceted, so we will only discuss algortihms and terms pertaining to the subject of this thesis.

Definition 21. *By Optuna trial or just trial we mean one run of the evolutionary algorithm with one chosen set of hyperparameters and the results of this run.*

Definition 22. *Optuna study a set of related trials with implied continuity of choice.*

Definition 23. *Optuna optimisation run describes run of the chosen optimisation algorithm, adding optimised trials to selected study.*

Note that study maybe composed of multiple optimisation runs. In this thesis we have only done one study per container and one optimisation run per study if not for anything htan to mind coputational time.

As the optimisation algorithm we have selected Tree-structured Parzen Estimator (TPE) due to the fairly low complexity and efectivness, wasting no trials as we want to conserve as much actual runtime as possible.

TPE

The Tree-structured Parzen Estimator (TPE) used in Optuna is a sequential model-based optimization method for hyperparameter tuning that reformulates Bayesian optimization in terms of density estimation over the input space rather than the objective space. It was introduced by Bergstra et al. (2011) as part of early work on scalable hyperparameter optimization Bergstra et al. [2011].

In a standard hyperparameter optimization setting, we observe a dataset of trials

$$\mathcal{D} = \{(x_i, y_i)\},$$

where x_i is a hyperparameter configuration and y_i is the corresponding loss (or score). TPE begins by splitting these observations into two groups using a threshold y^* , typically chosen as a quantile of observed outcomes:

- **Good configurations:** those with $y_i < y^*$
- **Bad configurations:** those with $y_i \geq y^*$

This split is not arbitrary—it defines what the algorithm currently considers promising based on past experience.

Instead of learning a direct predictive model $p(y \mid x)$, TPE learns two simpler distributions over configurations:

$$\ell(x) = p(x \mid y < y^*), \quad g(x) = p(x \mid y \geq y^*),$$

These distributions are typically estimated using smooth non-parametric density estimators (Parzen estimators), which can flexibly model continuous, discrete, and categorical hyperparameters without strong assumptions about the shape of the space. The key idea is that the next configuration should be one that is:

- likely under the “good” distribution, and
- unlikely under the “bad” distribution.

This leads to the selection rule:

$$x^* = \arg \max_x \frac{\ell(x)}{g(x)}.$$

Rather than optimizing this expression directly, TPE samples candidate points from $\ell(x)$ and evaluates them using this ratio, selecting the best candidate. This makes the method computationally efficient and avoids difficult global optimization of an acquisition function.

4.4.2 General Gridsearches

Now we will discuss grid searches that are common for all implementations of our EAs. Note that for convenience we do not present these concepts in the temporal order in which they have been evaluated.

Evaluation episodes vs. generations

Before we start other experiments we have to determine how many episodes will be used to evaluate an individual during the algorithm's run in these experiments. The parameter directly influences how much information the algorithm has extracted about the individual, as more episodes will lead to more indicative behavioural descriptors. However since the episodes take up non-trivial part of the computation we want to select the lowest possible number. For this purpose we will run a grid search comparing differential algorithm's performance over different number of generations. Since the Gymnasium environments discussed in chapter 2. are based on similar principles we will perform the search only on Cartpole but generalise the results to both. This is done to conserve time for more impactful experiments.

Generations grid search

During the refining in Bayesian search and Incremental grid search we use fixed number of generations for the optimisations. This means that we don't know how many generations are truly best. We perform a grid search over a range of generations. This will also allow us to compare performances and performance curves based on this parameter.

4.4.3 Incremental grid search

Because we obtained the optimised parameters from Bayesian search stochastically we would like to ensure that there is no easily obtainable set of parameters more performant than the set from BS. All however in time consuming fashion. To this end we have devised an algorithm of incremental grid search where we evaluate possibly useful deviations from the supposed local optima. Upon discovering new optima we recenter the search around it. If we are not successful in finding dominating set of parameters we end the search with the last obtained optima. We also end the search after more than 3 recentering (hops). Since this is only refining algorithm of already functional solution we do not expect long searches.

Algorithm 5: Incremental grid search general

Input: Initial argument set x_0 , number of hops H ,
evaluation algorithm A

Output: Final argument set x

```
 $x \leftarrow x_0;$ 
 $\mathcal{V} \leftarrow \emptyset;$ 
// fitness performances
 $s \leftarrow \emptyset;$ 
// final populations
 $pop \leftarrow \emptyset;$ 
 $s(x_0), diversity(x_0) \leftarrow A(x_0);$ 
for  $h \leftarrow 1$  to  $H$  do
    Generate valid candidate argument sets  $\mathcal{C} \leftarrow \text{Generate}(x, \mathcal{V}, h);$ 
    foreach  $c \in \mathcal{C}$  do
         $s(c), diversity(c) \leftarrow A(c);$ 
        // for  $c \in \mathcal{V}$  we use cached values
         $\mathcal{V} \leftarrow \mathcal{V} \cup \{c\};$ 
    if  $\mathcal{C} \neq \emptyset$  then
         $S^* \leftarrow \max_{c \in \mathcal{C}} s(c);$ 
         $\mathcal{B} \leftarrow \{c \in \mathcal{C} \mid s(c) = S^* \wedge s(x) \leq s(c)\};$ 
        if  $|\mathcal{B}| = 1$  then
             $x \leftarrow \text{only element of } \mathcal{B};$ 
        else if  $|\mathcal{B}| = 0$  then
            // We have not found improving solution
            return  $x, s(x);$ 
        else
             $x \leftarrow \arg \max_{c \in \mathcal{B}} (diversity(c));$ 
return  $x, s(x);$ 
```

The candidates are generated from neighbourhoods of the selected solution x in one or more hyperparameter. For hyperparameter H the neighbourhood is defined by δ_H . We generate $x_{-\delta}$, $x_{+\delta}$, x_0 such that for $x_{-\delta}(H) = x(H) - \delta_H$ and analogically for $x_{+\delta}$. The option of x_0 might not make sense to generate for a single hyperparameter as this is just x repeated but in combination of two or more hyperparameters this ensures we truly check all combinations consisting the neighbourhood of x in those hyperparameters.

4.4.4 $\mu + \lambda$ incremental grid search

For the purposes of $\mu + \lambda$ algorithm we split the Generation and evaluation of candidate solution for the iteration of the main loop into two parts. First part generates candidates in the neighbourhood on the hyperparameters $\mu + CR$, the second $\lambda + MR$. While all hyperparameters interact we claim that these pairs interact very strongly if not strongest as exploitation power of CR is directly interlinked with the stochastic diversity reservoir of the current population. On the other hand the size the new individual pool limits exploration power of mutation rate. It should be noted that the selection process happens after both candidate batch generations and so the candidates in $\lambda + MR$ stage might be generated from

solution already updated by $\mu + CR$ stage. The reason why we do this instead of generating all at once is time conservation and priority based on projected stability of the solution - we claim that alterations to the main population and crossover rate produce more stable algorithm based on preliminary experimentation. If we don't find the solution at first stage algorithm cannot prematurely terminate before the second stage is evaluated negatively as well.

4.4.5 DEs incremental grid search

As we described in section 4.4.4 we split the candidate generation even for the differential evolution. First stage alternates only size of the population. Second stage alterantes hyperparameters $\alpha + \gamma$ or the rescale factor and recombination rate introduced in 4. The algorithm again cannot terminate prematurely before both stages have been checked for improving solution.

4.5 Final comparison

In addition to graphical methods suggested in paper Beiranvand et al. [2017] to examine the results of the test we will use several statistical methods. Following advice in the paper García et al. [2009], which is specifically concerned with evolutionary algorithms, we will mainly utilise the non-parametrical tests which we introduce in this section. This enables us to compare results of our algorithm without imposing additional constraints.

The tests represent a null hypotheses H_0 which may or may not be rejected in favour of hypotheses H_1 . This is done based on obtained p-value p representing a probability of observing the obtained data assuming the null hypotheses is true. We define a threshold of significance α so that for $\alpha > p$ the null hypotheses is rejected. Usually values $\alpha = 0.1$ or $\alpha = 0.05$ are chosen.[García et al., 2009]

4.5.1 Friedman test

We will use description of Friedman test from García et al. [2009]. There it is described as a statistical test testing k algorithms. The null hypothesis is that all k algorithm sample from the same population distribution. If rejected we accept at least one of the k algorithms samples from a different population distribution from the rest. We compute rankings r_j^i of each algorithm j for each comparison instance i . In the case of this thesis comparison instance is a initial coditions set for an particular instance of evolutionary algorithm. Rank is distributed from the best having rank 1 to the worst rank k . Then we compute the Friedman statistic χ_F^2 :

$$\chi_F^2 = \frac{12N}{k(k+1)} \left(\sum_{j=1}^k R_j^2 - \frac{k(k+1)^2}{4} \right)$$

where $R_j = \frac{1}{N} \sum_{i=1}^N r_j^i$ and N i the number of comparison instances. The χ_F^2 is then converted to p-value through a combinatorial computation. In reviewed literature it is not common to go over the computational process explicitly as it maybe quite complex in the variety of aproximations. We will use the implementation in Python library SciPy which cites Spencer et al. [2025].

4.5.2 Wilcoxon signed-rank test

Introduced in the paper Wilcoxon [1992] in both paired and unpaired version, Wilcoxon signed-rank test enables us to compare two algorithm without assumptions about the distributions of their tested output. We start by computing differences of the algorithms' A and B performances in each instance i of total of N like so:

$$d_i = A_i - B_i$$

Unlike in Friedman test where the rank is distribute across algorithms, in Wilcoxon test distributes rank across the absolute value of the differences $|d_i|$. Then we compute sums of ranks testifying for each of the compared algorithm. R^+ is the sum of ranks where algorithm A won ($d_i > 0$) while R^- is the sum of ranks where algorithm B won ($d_i < 0$). For $d_i = 0$ we split the ranks between the two sums [García et al., 2009].

$$R^+ = \sum_{d_i > 0} rank(d_i) + \frac{1}{2} \sum_{d_i = 0} rank(d_i)$$

$$R^- = \sum_{d_i < 0} rank(d_i) + \frac{1}{2} \sum_{d_i = 0} rank(d_i)$$

According to the paper García et al. [2009] we then obtain value

$$T = \min(R^+, R^-)$$

The T is then compared to the value of Wilcoxon distribution with N degrees of freedom. The null hypothesis that both algorithms perform equally is rejected if the T is less or equal. Corresponding p-value is calculated using normal approximation of Wilcoxon T statistic. [Cureton, 1967]

Holm's and Hochberg adjustment

Since we will be performing number of comparisons between different algorithm instances, we need to take measures to prevent family-wise error rate (FWER). To do so García et al. [2009] recommends Holm's and Hochbergs adjustment procedures.

Holm's procedure orders the obtained p-values so that: $p_1 \leq p_2 \leq \dots \leq p_{k-1}$. Each p-value p_i , starting with the most significant, is then compared to $\frac{\alpha}{k-i}$ where α is significance threshold. We continue up the chain as long as we can reject the null hypothesis H_i based on $\frac{\alpha}{k-i}$. We stop once we cannot reject the null hypothesis and we conclude that all subsequent null-hypotheses hold up. [García et al., 2009]

In Hochberg's procedure we start from the least significant p-value and work our way down the chain. For the chain $p_1 \geq p_2 \geq \dots \geq p_{k-1}$ comparing p_i with $\frac{\alpha}{i}$. If the null hypotheses hold's up we move on to the next p-value. If it is rejected we consider all subsequent null-hypotheses rejected as well.[García et al., 2009]

5. Experiments

In this chapter we will discuss the experiment and their results. The code for the experiments has been written in Python 3.10 and experiments were conducted on several computers characterised in attachment ???. The complete project with data and scripts is available publicly in Github repository.

5.1 Bayesian Search

As mentioned in section 4.4.1, we perform one optimisation run per container, resulting in one relevant study. This is purely practical measure pertaining to time feasibility of the experiments. In each optimisation run we perform fixed number of trials. We use 150 trials as baseline, adding 10-20 trials for more parametrised containers. Because we are dealing with stochastic method this number does not guarantee neither good or bad performance and has been arbitrarily chosen from experience with preliminary experiments. The hyperparameter selection for each run is done within limits detailed in table ??.

5.1.1 Iterative grid search

5.1.2 Generational grid search

5.2 Final comparison

5.2.1 Cartpole

5.2.2 Lunar Lander

5.3 Discussion

Conclusion

I have successfully developed and implemented framework for testing evolutionary strategies and differential evolution in several setups, such as fitness, novelty and their respective archiving and a combination of them. I have also proposed behavioral models for both environments Lunar Lander and Cartpole. These behavioral models are a modest success, while Lunar Lander model, unlike Cartpole, does not consistently achieve solution grade score, it is still very well-behaved. I have also proposed a hyperparameter searching algorithm based on Bayesian search. While the search itself is rather limited, this reflects the conditions of the used environments, which are time-exhausting to simulate in large numbers. The algorithm allows us to compare the EAs on their performance given hyperparameters found by this search. The areas of comparisons were: absolute performance, rate of convergence and diversity of the final populations.

Bibliography

- Vahid Beiranvand, Warren Hare, and Yves Lucet. Best practices for comparing optimization algorithms. *Optimization and Engineering*, 18(4):815–848, 2017. ISSN 1573-2924. doi: 10.1007/s11081-017-9366-1. URL <http://dx.doi.org/10.1007/s11081-017-9366-1>.
- James S. Bergstra, Rémi Bardenet, Yoshua Bengio, and Balázs Kégl. Algorithms for hyper-parameter optimization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2011.
- Hans-Georg Beyer and Hans-Paul Schwefel. Evolution strategies - a comprehensive introduction. *Natural Computing*, 1:3–52, 03 2002. doi: 10.1023/A:1015059928466.
- Giuseppe Cuccu and Faustino Gomez. When novelty is not enough. volume 6624, pages 234–243, 04 2011. ISBN 978-3-642-20524-8. doi: 10.1007/978-3-642-20525-5_24.
- Edward E. Cureton. The normal approximation to the signed-rank sampling distribution when zero differences are present. *Journal of the American Statistical Association*, 62(319):1068–1069, 1967. doi: 10.1080/01621459.1967.10500917. URL <https://www.tandfonline.com/doi/abs/10.1080/01621459.1967.10500917>.
- Stephane Doncieux, Alban Laflaquière, and Alexandre Coninx. Novelty search: a Theoretical Perspective. In *GECCO '19: Genetic and Evolutionary Computation Conference*, pages 99–106, Prague Czech Republic, France, July 2019. ACM. doi: 10.1145/3321707.3321752. URL <https://hal.science/hal-02561846>.
- A. Eiben and Jim Smith. *Introduction To Evolutionary Computing*, volume 45. 01 2003. ISBN 978-3-642-07285-7. doi: 10.1007/978-3-662-05094-1.
- Salvador García, Daniel Molina, Manuel Lozano, and Francisco Herrera. A study on the use of non-parametric tests for analyzing the evolutionary algorithms' behaviour: A case study on the cec'2005 special session on real parameter optimization. *J. Heuristics*, 15:617–644, 12 2009. doi: 10.1007/s10732-008-9080-4.
- David E. Goldberg and Kalyanmoy Deb. A comparative analysis of selection schemes used in genetic algorithms. volume 1 of *Foundations of Genetic Algorithms*, pages 69–93. Elsevier, 1991. doi: <https://doi.org/10.1016/B978-0-08-050684-5.50008-2>. URL <https://www.sciencedirect.com/science/article/pii/B9780080506845500082>.
- Jorge Gomes, Pedro Mariano, and Anders Lyhne Christensen. Devising effective novelty search algorithms: A comprehensive empirical study. In *Proceedings of the 2015 Annual Conference on Genetic and Evolutionary Computation, GECCO '15*, page 943–950, New York, NY, USA, 2015. Association for Computing Machinery. ISBN 9781450334723. doi: 10.1145/2739480.2754736. URL <https://doi.org/10.1145/2739480.2754736>.

- Leslie Pack Kaelbling, Michael L. Littman, and Anthony R. Cassandra. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1):99–134, 1998. ISSN 0004-3702. doi: [https://doi.org/10.1016/S0004-3702\(98\)00023-X](https://doi.org/10.1016/S0004-3702(98)00023-X). URL <https://www.sciencedirect.com/science/article/pii/S000437029800023X>.
- Joel Lehman and Kenneth Stanley. Abandoning objectives: Evolution through the search for novelty alone. *Evolutionary computation*, 19:189–223, 06 2011. doi: 10.1162/EVCO_a_00025.
- Elliot Meyerson, Joel Lehman, and Risto Miikkulainen. Learning behavior characterizations for novelty search. In *Proceedings of the Genetic and Evolutionary Computation Conference 2016*, GECCO '16, page 149–156, New York, NY, USA, 2016. Association for Computing Machinery. ISBN 9781450342063. doi: 10.1145/2908812.2908929. URL <https://doi.org/10.1145/2908812.2908929>.
- Stéphane Ross, Joelle Pineau, Sébastien Paquet, and Brahim Chaib-draa. Online planning algorithms for pomdps. *CoRR*, abs/1401.3436, 2014. URL <http://arxiv.org/abs/1401.3436>.
- Stuart Russell and Peter Norvig. *Artificial Intelligence, Global Edition A Modern Approach*. Pearson Deutschland, 2021. ISBN 9781292401133. URL <https://elibrary.pearson.de/book/99.150005/9781292401171>.
- Achkan Salehi, Alexandre Coninx, and Stephane Doncieux. Br-ns: an archive-less approach to novelty search. In *Proceedings of the Genetic and Evolutionary Computation Conference*, GECCO '21, page 172–179. ACM, 2021. doi: 10.1145/3449639.3459303. URL <http://dx.doi.org/10.1145/3449639.3459303>.
- Neil Spencer, Nigel Smeeton, and Peter Sprent. *Applied Nonparametric Statistical Methods*. 03 2025. ISBN 9780429326172. doi: 10.1201/9780429326172.
- Rainer Storn and Kenneth Price. Differential evolution - a simple and efficient heuristic for global optimization over continuous spaces. *Journal of Global Optimization*, 11:341–359, 01 1997. doi: 10.1023/A:1008202821328.
- Frank Wilcoxon. *Individual Comparisons by Ranking Methods*, pages 196–202. Springer New York, New York, NY, 1992. ISBN 978-1-4612-4380-9. doi: 10.1007/978-1-4612-4380-9_16. URL https://doi.org/10.1007/978-1-4612-4380-9_16.

List of Figures

1.1	Agent and enviroment realtionship	4
4.1	Structure of Carpole agentic network	22
4.2	Structure of Lunar Lander agentic network	23

List of Tables

2.1	Observation space of Cartpole	9
2.2	Cartpole Action Space.	9
2.3	Observation space of Lunar Lander	10
2.4	Action space definition for the control system.	11

List of Abbreviations

EA	Evolutionary Algorithm
ES	Evolutionary Strategy
DE	Differential Evolution
BS	Bayesian Search

A. Attachments

A.1 Experiment Data Format

A.1.1 Single Run Statistics

A.1.2 Adaptive Grid Search Protocol

A.1.3 Generations Grid Protocol

A.1.4 Final Experiment Protocol

A.2 Used Hardware